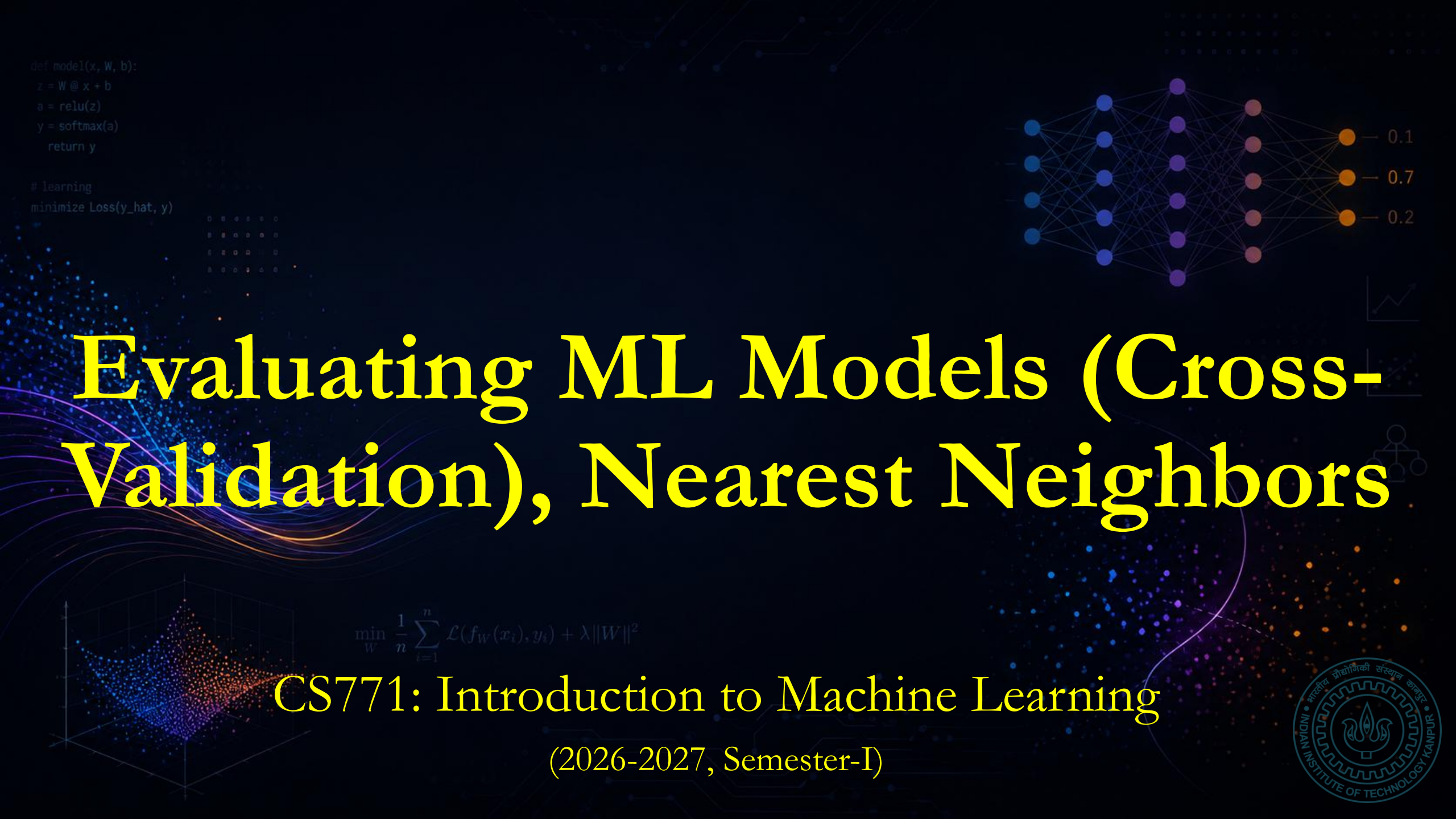


Evaluating ML Models (Cross-Validation), Nearest Neighbors

CS771: Introduction to Machine Learning

(2026-2027, Semester-I)

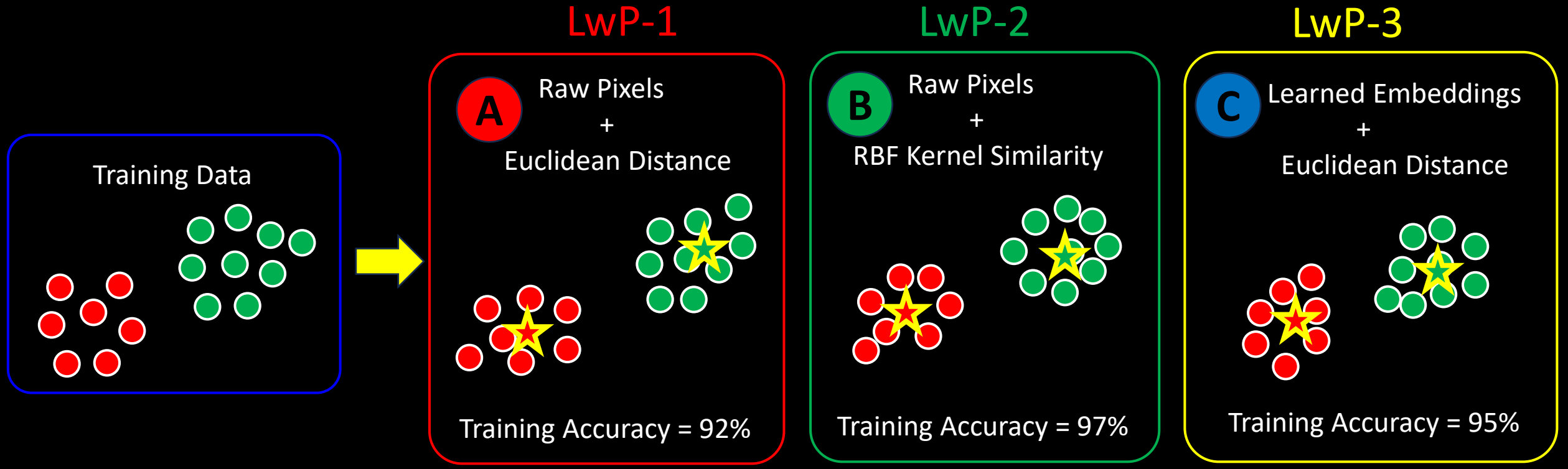
Plan Today

- How to evaluate how well a model will do on future data
  - Cross-validation
- Nearest neighbors for supervised learning
- Other applications of nearest neighbors



ML Model trained with different configurations

Here: Model = LwP, configuration = which features and which distance/similarity



Best on training data ≠ best on future data

We need unseen examples to choose among configurations.

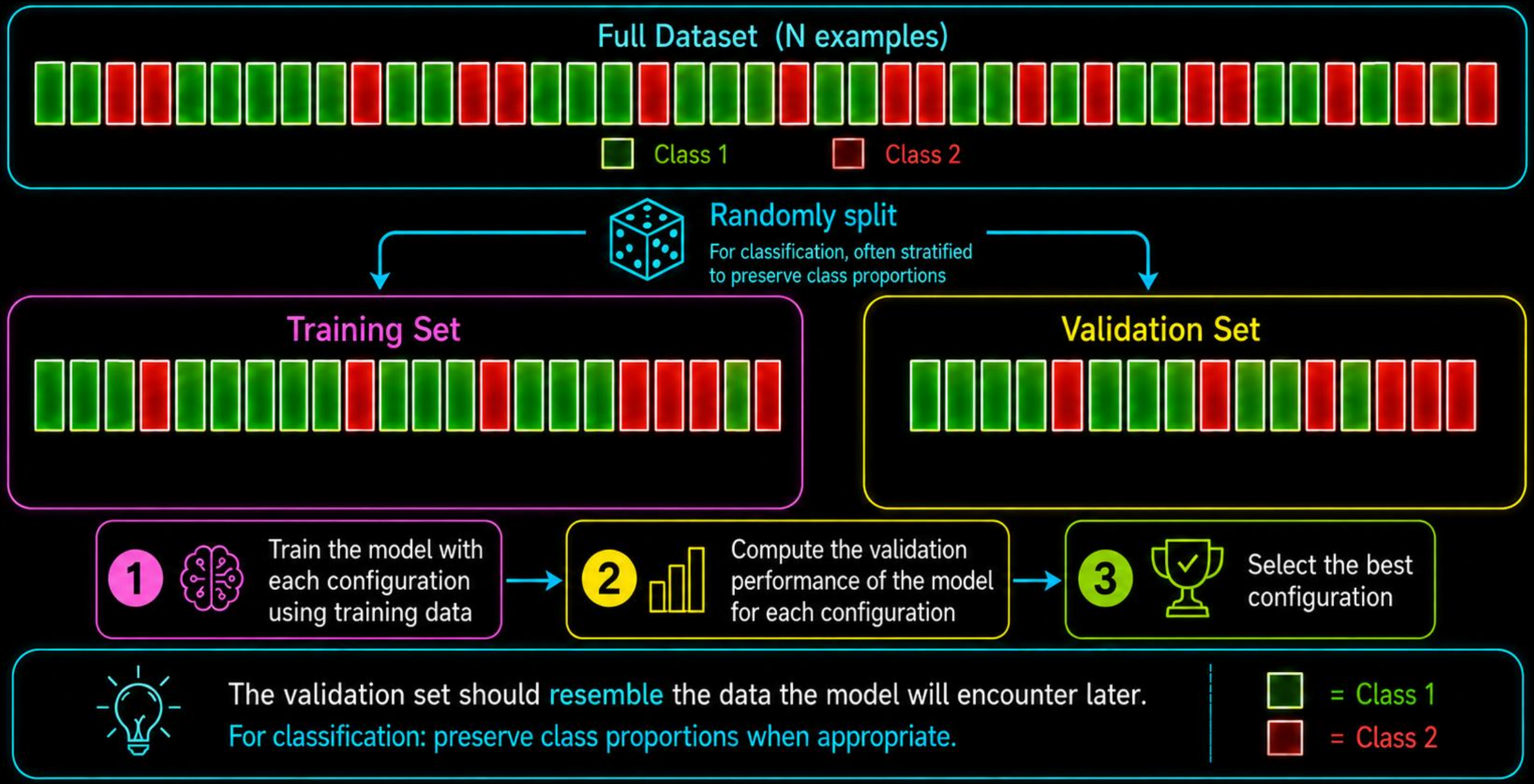
Training accuracy measures performance on examples the model has already seen.

So we can't rely on training accuracy to pick the best configuration.

We need something that can serve as a proxy of unseen future test data to choose among configurations.

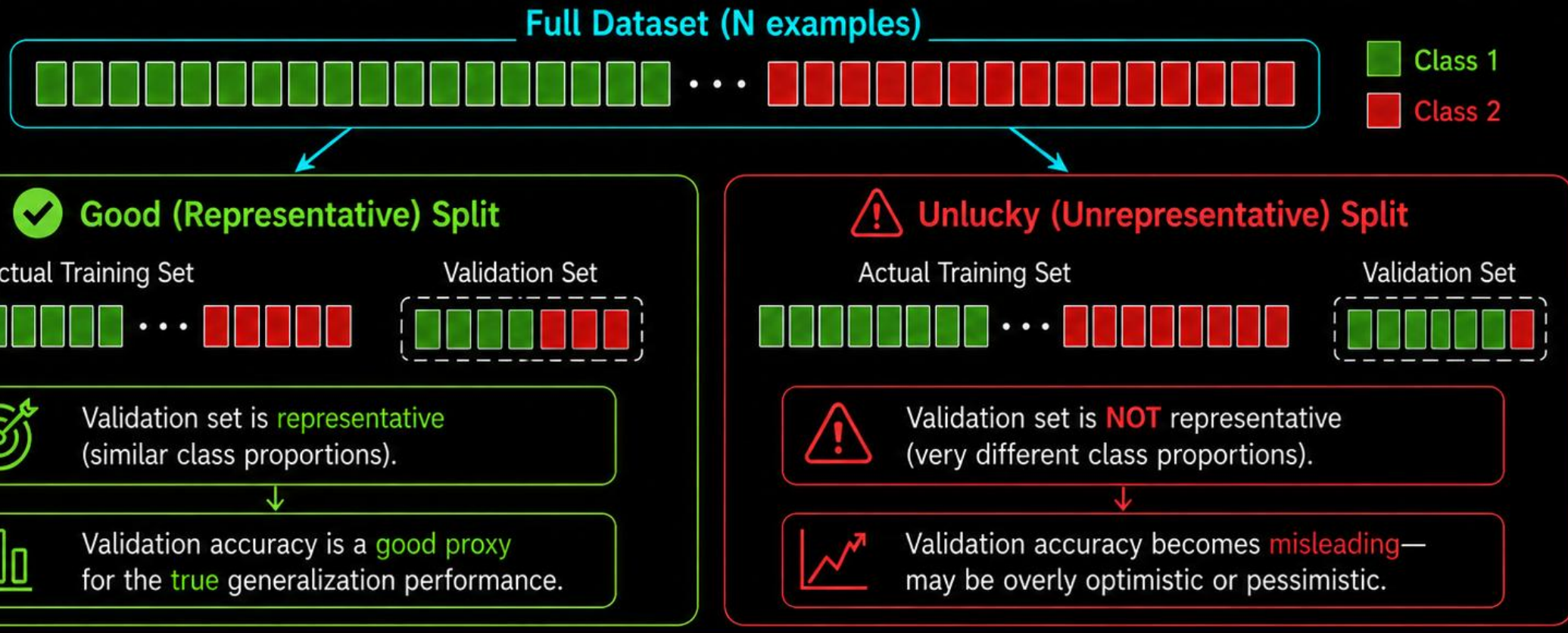
Goal: Choose the configuration that is expected to perform best on unseen future test data.

Validation Set: A Proxy for Unseen Test Data



Relying on a Single Validation Set May Be Unlucky

We randomly split the data into a training set and a validation set by "stealing" a part of the training data.



Different random splits may produce different validation accuracies.
A single validation split may give an unreliable estimate of model performance.

Can we reduce this dependence on one lucky (or unlucky) split?

Better Evaluation: *K*-Fold Cross-Validation (CV)

Instead of relying on one random validation split, we use **K** different splits to get a more reliable estimate.

1 Partition the data into K folds

Full Dataset (N examples)



Split into K non-overlapping folds

Fold 1 | Fold 2 | Fold 3 | ... | Fold K



The folds are:

- ✓ Non-overlapping
- ✓ Their union is the entire dataset

$$\bigcup_{i=1}^{K} \text{Fold}_i = \text{Full Dataset}$$

Class 1 Class 2

2 Perform K iterations

Iteration	Fold 1	Fold 2	Fold 3	...	Fold K
1	V	T	T		T
2	T	V	T		T
3	T	T	V		T
...	...	...	...		...
K	T	T	T		V

T Training fold V Validation fold

In each iteration, one fold is used for validation and the remaining $K - 1$ folds for training.

3 Compute and average performance

Iteration 1 → score_1
Iteration 2 → score_2
Iteration 3 → score_3
...
Iteration K → score_K

$$\text{Average CV score} = \frac{1}{K}\sum_{i=1}^{K} \text{score}_i$$

This average is our cross-validation estimate of generalization performance.

4 Select the best configuration

For each configuration (hyperparameter setting), repeat the K iterations and compute the average CV score.

Configuration	Average CV Score
Config A	0.842
Config B	**0.918**
Config C	0.867
...	...

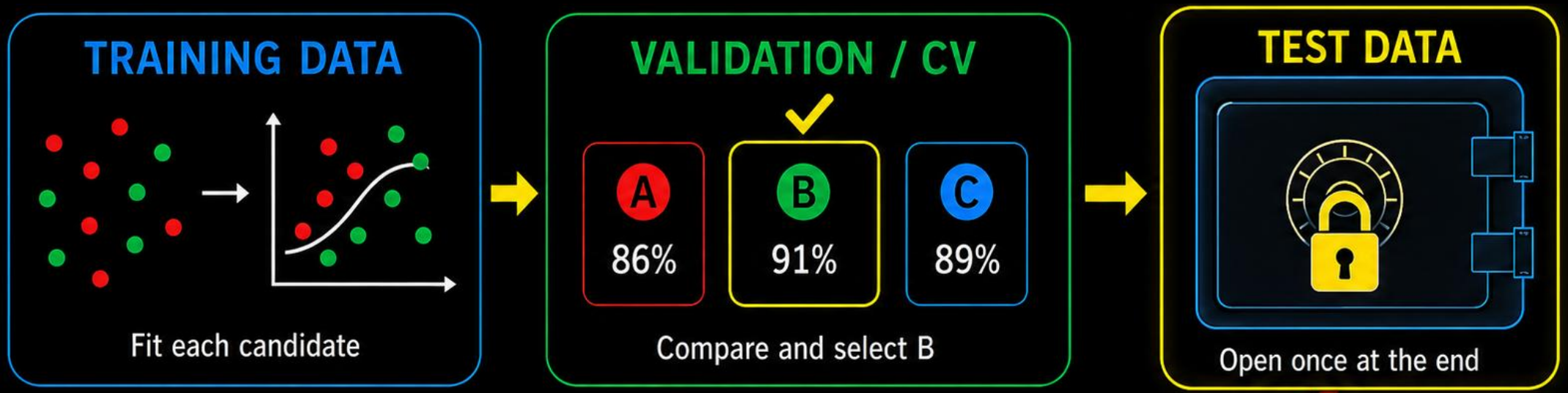
Choose the configuration with the best (highest) average CV score.

Why it helps?

- ✓ Reduces dependence on any one lucky (or unlucky) split.
- ✓ Uses (almost) all data for training and validation.
- ✓ Provides a more reliable estimate of generalization.

Caution: No Peeking Into Test Data During Training

Never use the test set to train the model or for doing cross-validation



If the test result changes your choice, it is no longer a test set.

CARDINAL SIN!

TRAIN to fit. **VALIDATE** to choose. **TEST** to report.

⚠ Repeatedly checking the test set also leaks information.

Cross-Validation is Largely Automated Nowadays

Thanks to ML libraries, most steps of CV are automated (just function calls)

1 Describe the configurations

- A raw pixels + Euclidean
- B raw pixels + RBF similarity
- C learned embeddings + Euclidean

2 The library runs CV

```
from sklearn.model_selection
  import cross_val_score

for cfg in configurations:
   model = make_LwP(cfg)
   scores[cfg] = cross_val_score(
     model, X_train, y_train, cv=5
   ).mean()

best_cfg = max(scores, key=scores.get)
```

make_LwP(cfg) is a user-defined constructor (not a scikit-learn function).

fit → validate → rotate → average

Fold 1	VALID	TRAIN	TRAIN	TRAIN	TRAIN
Fold 2	TRAIN	VALID	TRAIN	TRAIN	TRAIN
Fold 3	TRAIN	TRAIN	VALID	TRAIN	TRAIN
Fold 4	TRAIN	TRAIN	TRAIN	VALID	TRAIN
Fold 5	TRAIN	TRAIN	TRAIN	TRAIN	VALID

3 Select by mean CV score

- A 86%
- B 91% ✓ selected
- C 89%

Then refit the selected configuration on all training data.

The same pattern works for many hyperparameters



Automation executes the procedure; we still choose the metric, candidates, and splitting scheme.

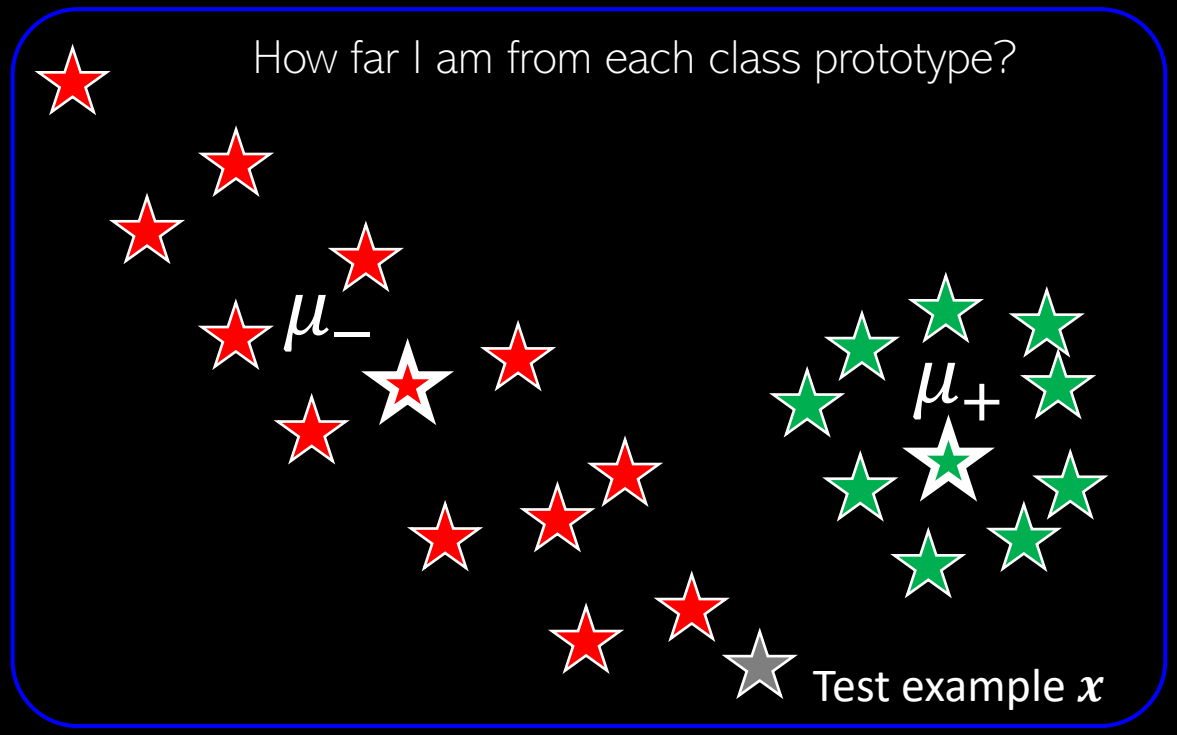
Our Second ML Algorithm

(Like LwP uses only distance/similarity computations)



Global Geometry vs Local Geometry?

Instead of comparing with class prototypes, how about comparing with individual training inputs and checking the label of the nearest training input(s)?





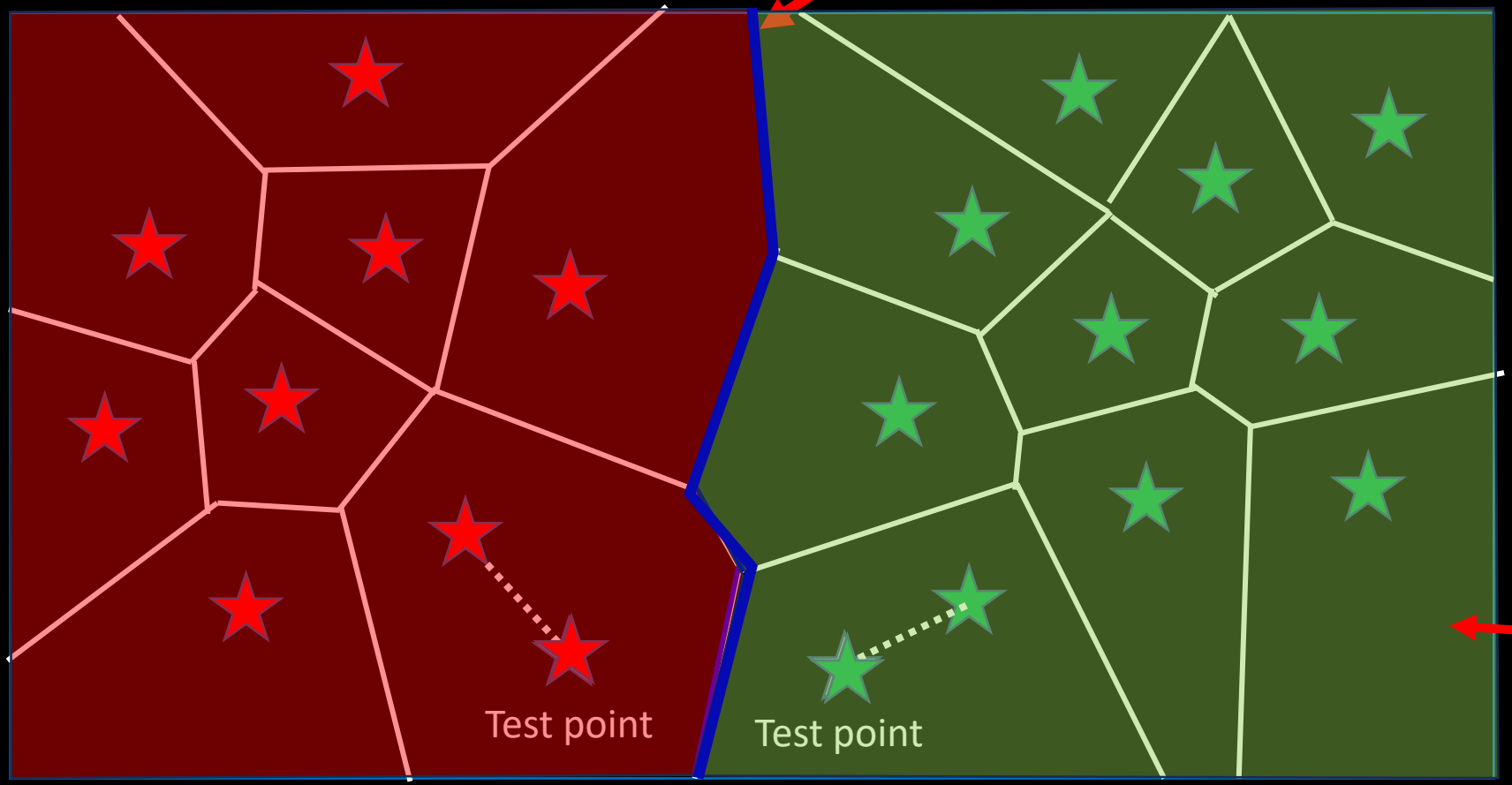
Looking at neighboring inputs can give us idea about local structure or geometry of the data which in many cases be more helpful than the idea of global structure or geometry that prototypes give us (like for the example above)

1-Nearest Neighbor Classification

Interesting. Even with Euclidean distances, it can learn nonlinear decision boundaries?



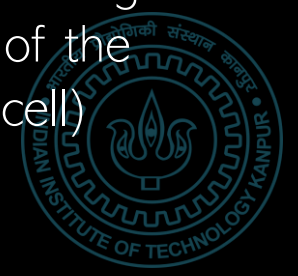
Decision boundary



Indeed. And that's possible since NN is a "local" method (looks at a local neighborhood of the test point to predict its output/label)

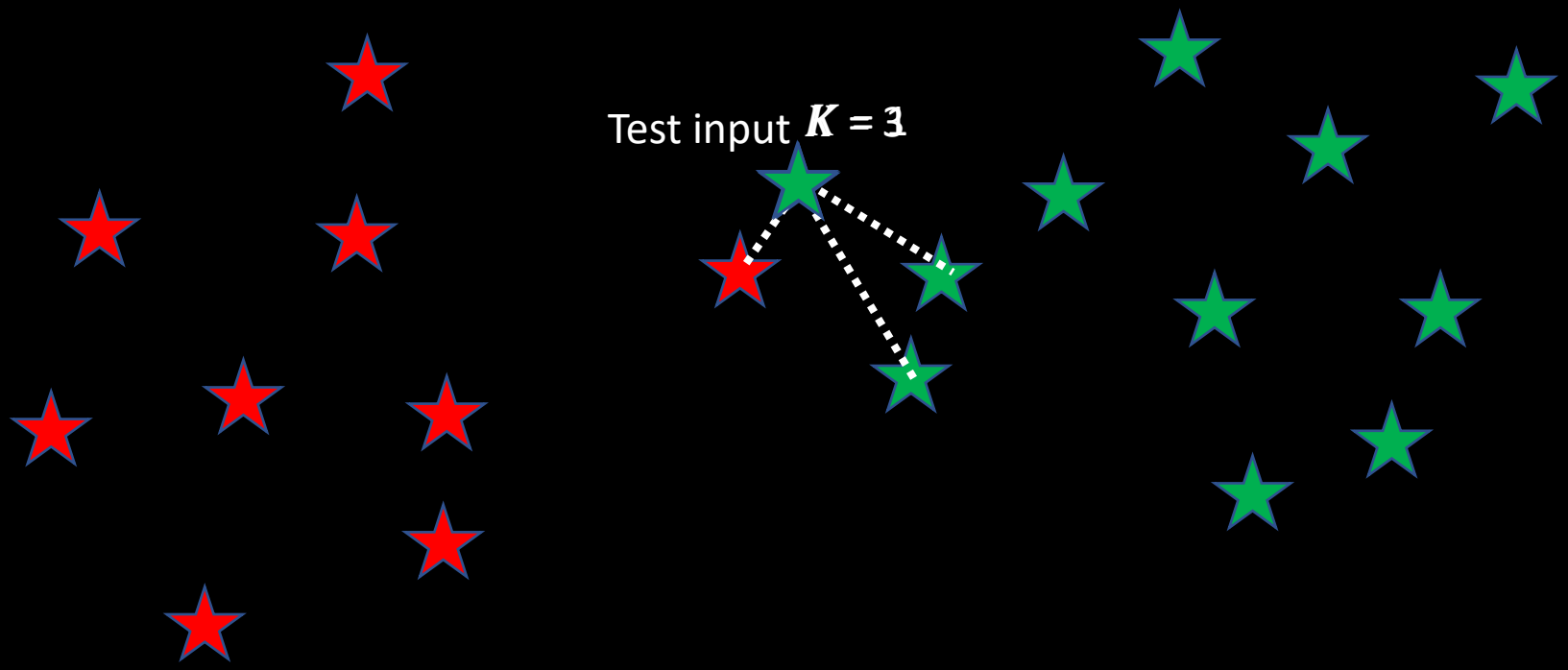


Nearest neighbour approach induces a Voronoi tessellation/partition of the input space (all test points falling in a cell will get the label of the training input in that cell)



K-Nearest Neighbors (*K*-NN) Classification

- In many cases, it helps to look at not one but $K > 1$ nearest neighbors





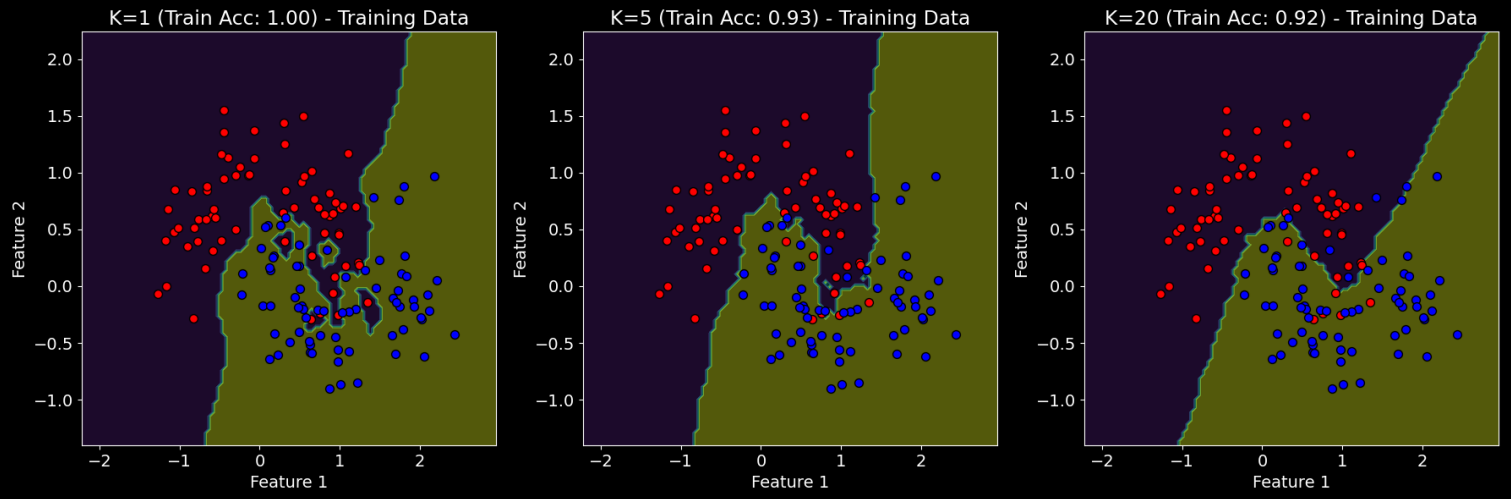
How to pick the "right" K value?

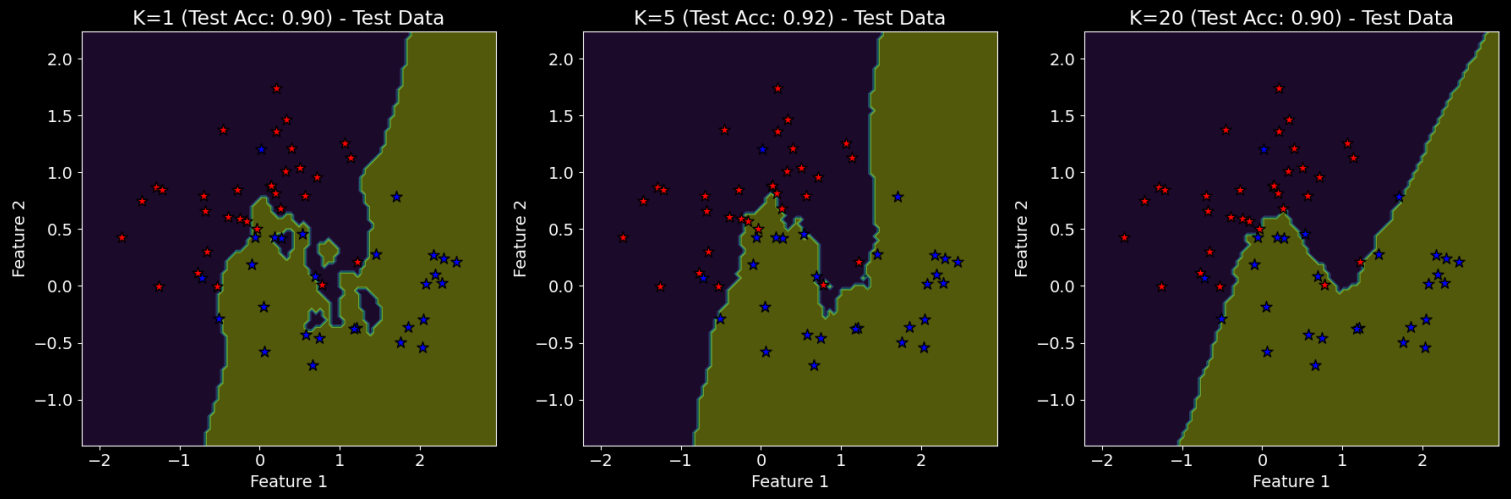


K is this model's "hyperparameter". One way to choose it is using cross-validation

- Essentially, taking more votes helps!
  - Helps deal with outlier training inputs
  - Also helps prevent overfitting on training data (more on this later)

K-NN Classifn: Shapes of Decision Boundaries



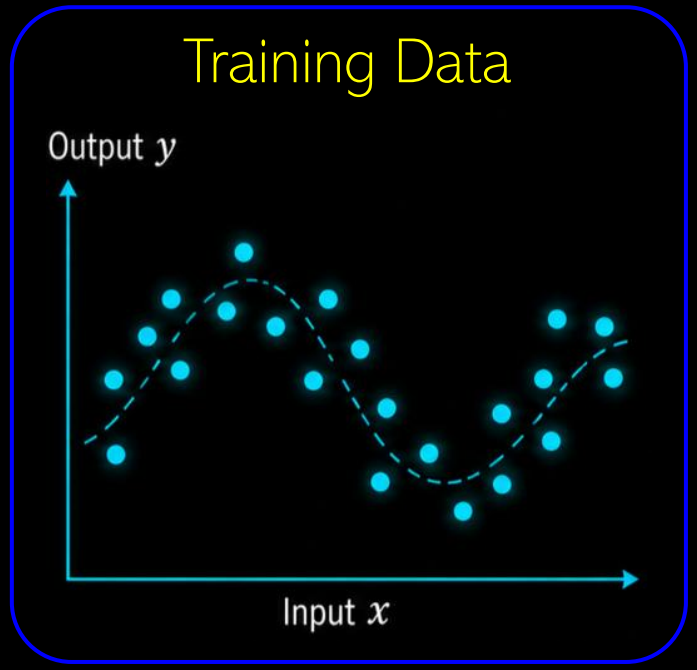


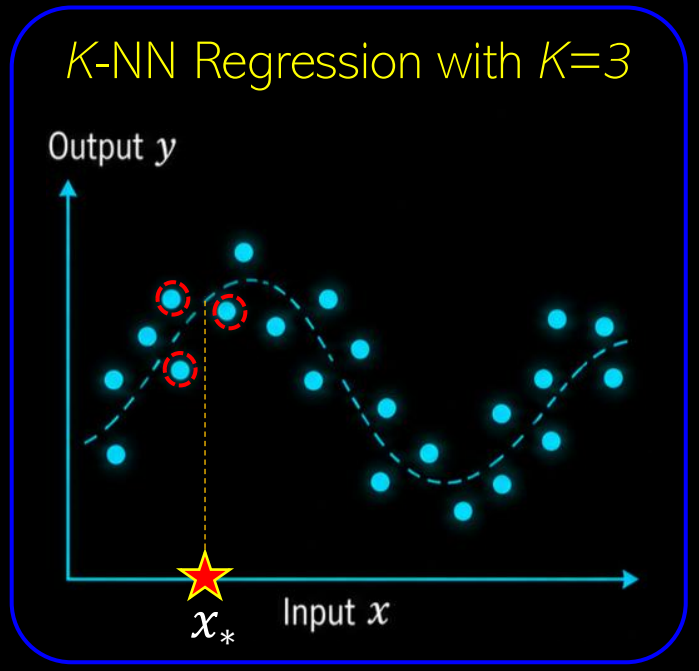
- As *K* varies, the shape of decision boundary changes
- Small *K* usually leads to more complex decision boundaries and high training accuracies
  - Usually a sign of overfitting on the training data
- Larger *K* usually leads to smoother decision boundaries
- There is a sweet-spot (not too small, not too large) for *K* that yields the best validation/test set performance

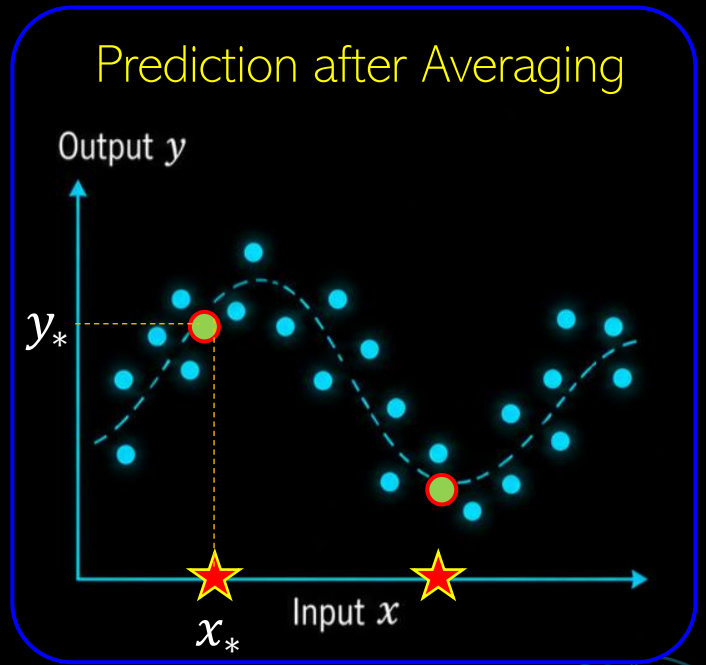


K-NN for Regression

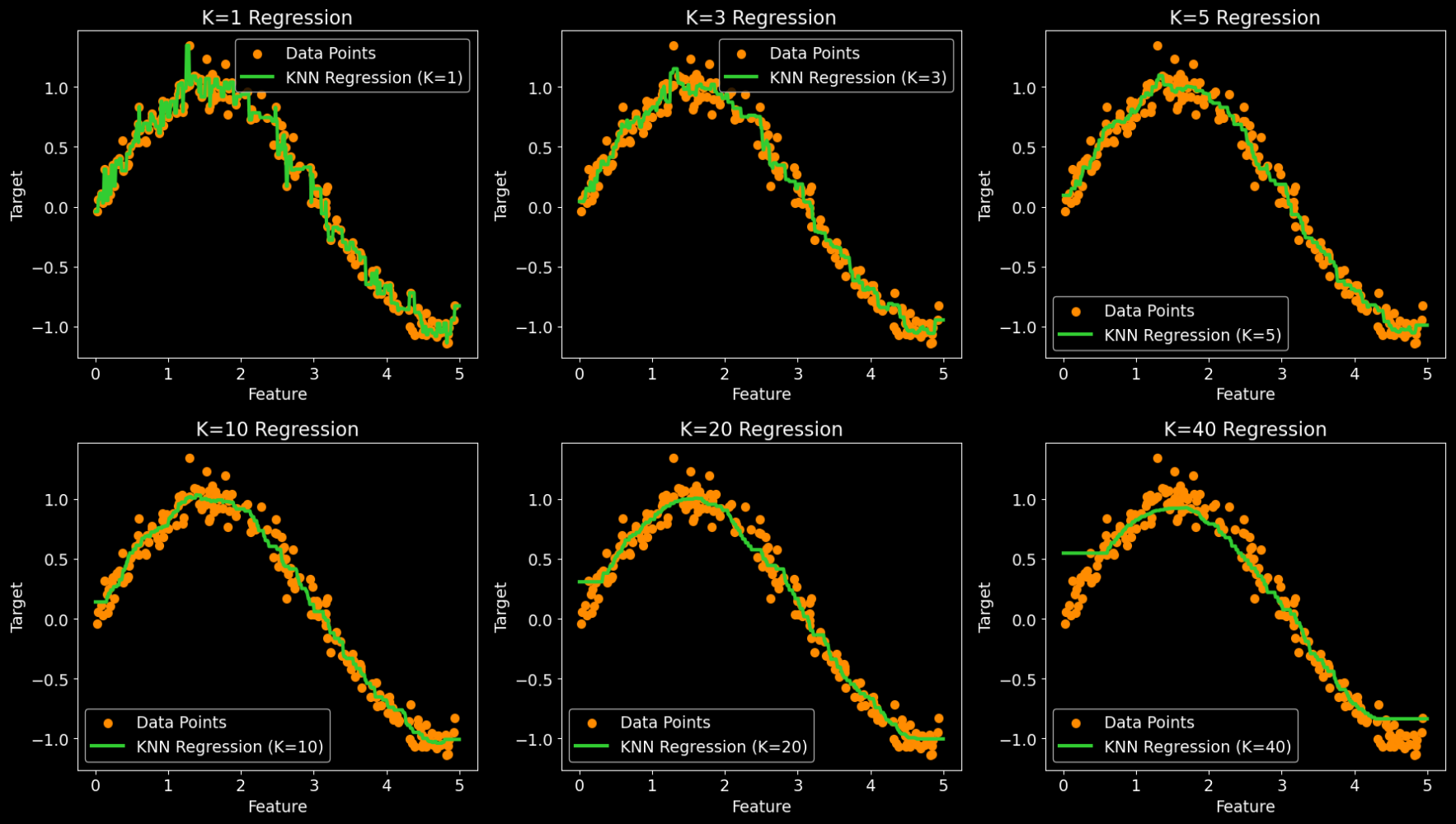
- Predicted response is an average of the responses of nearest neighbors







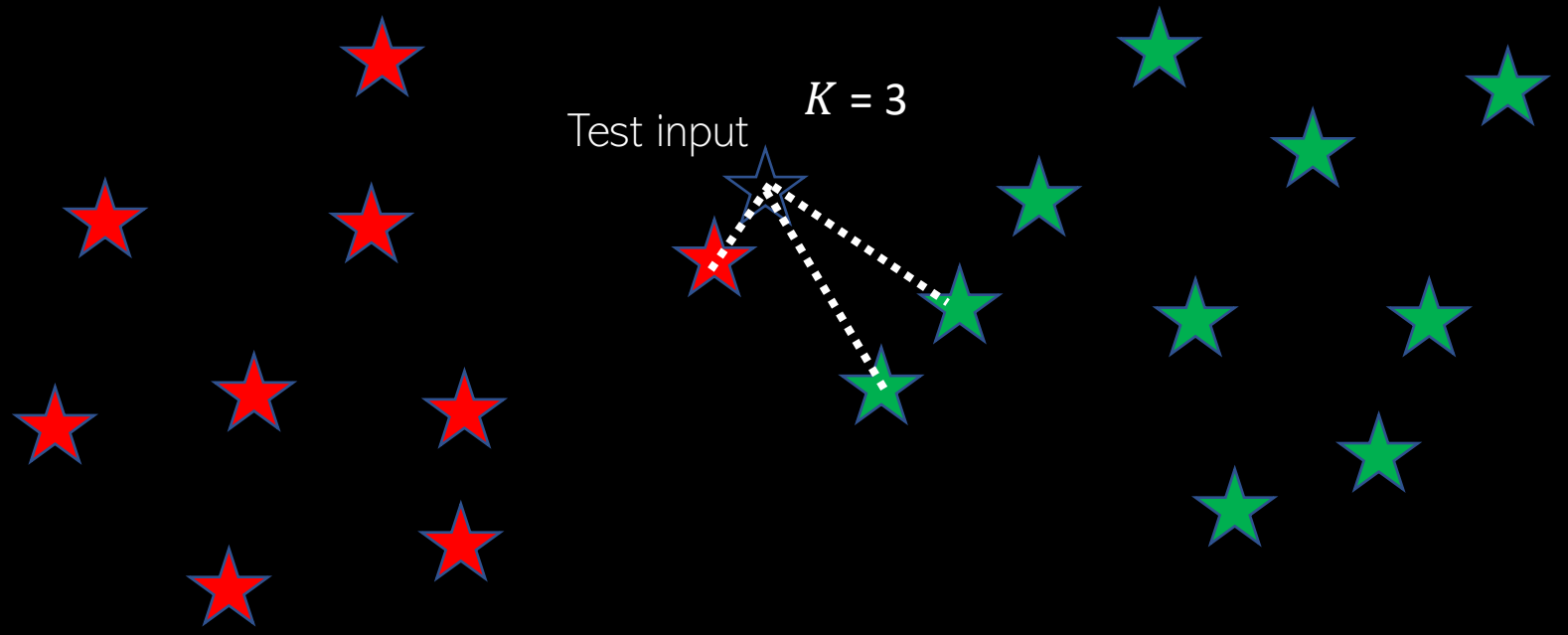
K-NN Regression: Shapes of Learned Curves



- Very small *K*: non-smooth curves (potential overfitting on training data)
- Very large *K*: too smooth curves
- Moderate *K*: The Sweet-spot (good validation/test set performance)

Distance-weighted *K*-Nearest Neighbors

- The standard KNN treats all nearest neighbors equally (all vote equally)



- An improvement: When voting, give more importance to closer training inputs

Unweighted KNN prediction: $\frac{1}{3}$ ★ + $\frac{1}{3}$ ★ + $\frac{1}{3}$ ★ = ★

Weighted KNN prediction: $\frac{3}{5}$ ★ + $\frac{1}{5}$ ★ + $\frac{1}{5}$ ★ = ★

In weighted approach, a single red training input is being given 3 times more importance than the other two green inputs since it is sort of "three times" closer to the test input than the other two green inputs



K-NN Prediction Rule, Mathematically

- Let's denote the set of K nearest neighbors of a test input $\boldsymbol{x}_*$ by $N_K(\boldsymbol{x}_*)$
- Predicted label, for unweighted *K*-NN is given by

Average of the responses of the *K* nearest neighbors

Regression: $$y_* = \frac{1}{K} \sum_{n \in N_K(\boldsymbol{x}_*)} y_n$$

"mode" returns the majority label of the *K* nearest neighbors

Classification: $$y_* = \text{mode}\,\{y_n : n \in N_K(\boldsymbol{x}_*)\}$$

- For weighted *K*-NN, we can define a weight for each response y_n

τ is a tunable hyperparameter

$w_n(\boldsymbol{x}_*)$ will be large if $\boldsymbol{x}_n$ is close to the test input $\boldsymbol{x}_*$

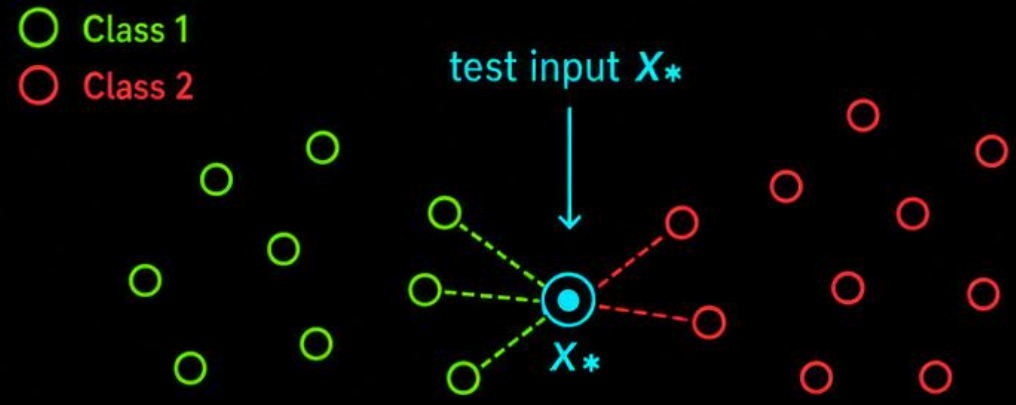
$$w_n(\boldsymbol{x}_*) = \frac{\exp[-d(\boldsymbol{x}_*, \boldsymbol{x}_n)/\tau]}{\sum_{n \in N_K(\boldsymbol{x}_*)} \exp[-d(\boldsymbol{x}_*, \boldsymbol{x}_n)/\tau]}$$

$d(\boldsymbol{x}_*, \boldsymbol{x}_n)$ is the distance between $\boldsymbol{x}_n$ and $\boldsymbol{x}_*$

... and use these weights in the averaging or mode operation (exercise: write the weighted versions of the prediction rules for regression and classification)



Nearest Neighbors: Some Comments



find the k nearest stored examples ($k = 5$)

majority vote → Class 1

Memory-based
The training examples are retained and consulted at prediction time.

Instance-based
A new input is predicted from nearby stored instances—not from a separately fitted predictive model.

Non-parametric
There is no fixed-size parameter vector whose dimension stays independent of N; effective model complexity can grow with the data.

k and the distance/similarity function are still hyperparameters.

What happens at prediction time?

Store N training vectors → Receive test input X_* → Compute N distances / similarities → Select k neighbors → Vote or average

N comparisons × $O(D)$ per comparison = $O(ND)$ per test input

Naïve brute-force search for N stored D-dimensional vectors

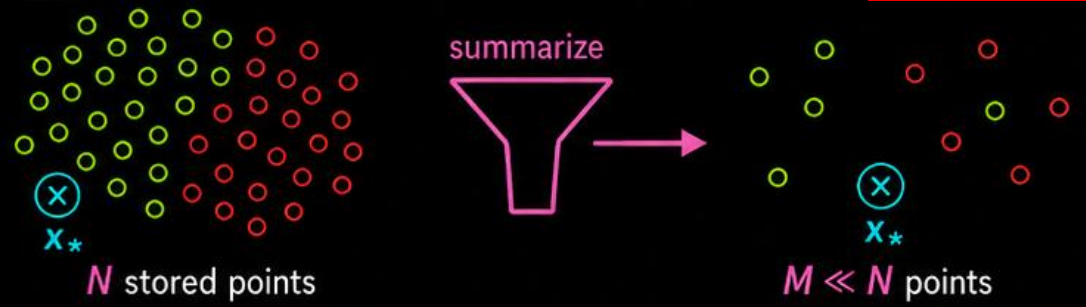
Indexes and approximate search can reduce retrieval time

Stored vectors require $O(ND)$ memory

Making Nearest Neighbors More Efficient

1 Keep fewer reference points

LwP is an extreme case (class means become the reference points)



Fewer stored points → fewer distance computations

Reduce N

2 Use lower-dimensional embeddings



Same N comparisons; each comparison is cheaper

$O(D) \rightarrow O(d)$ per comparison

3 Encode images as binary vectors

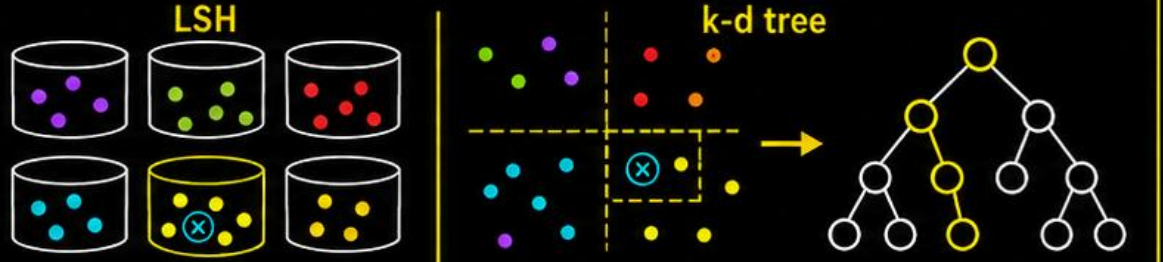


Bit operations make comparisons very fast

compact storage + fast comparison

4 Divide and conquer

Partition or index the space; search only promising regions

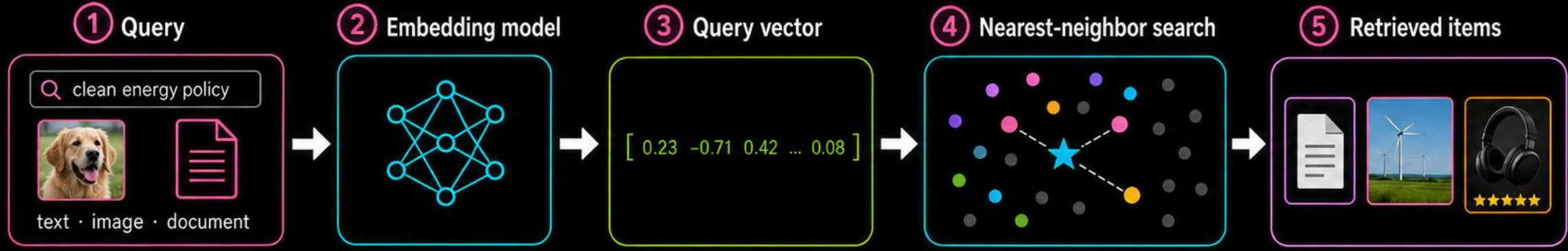


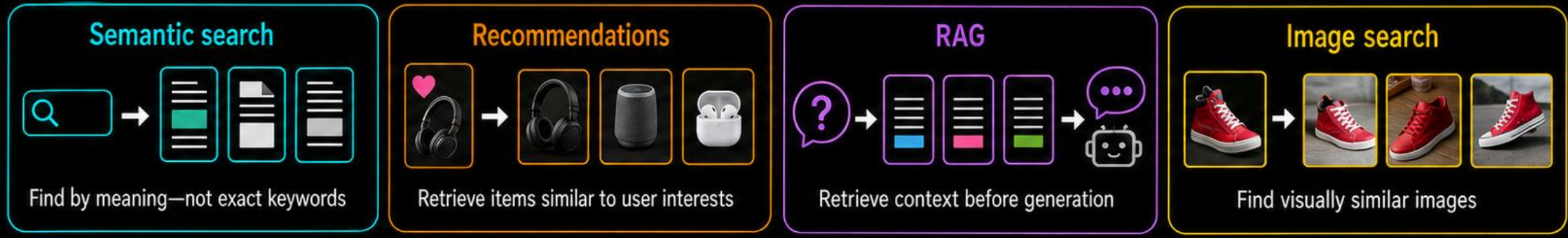
Avoid a full scan

Speed comes from reducing N, reducing dimension, using cheaper codes, or pruning the search.

Nearest Neighbors: Apps Beyond Sup. Learning

Similarity Search & Retrieval problems require finding nearest neighbors





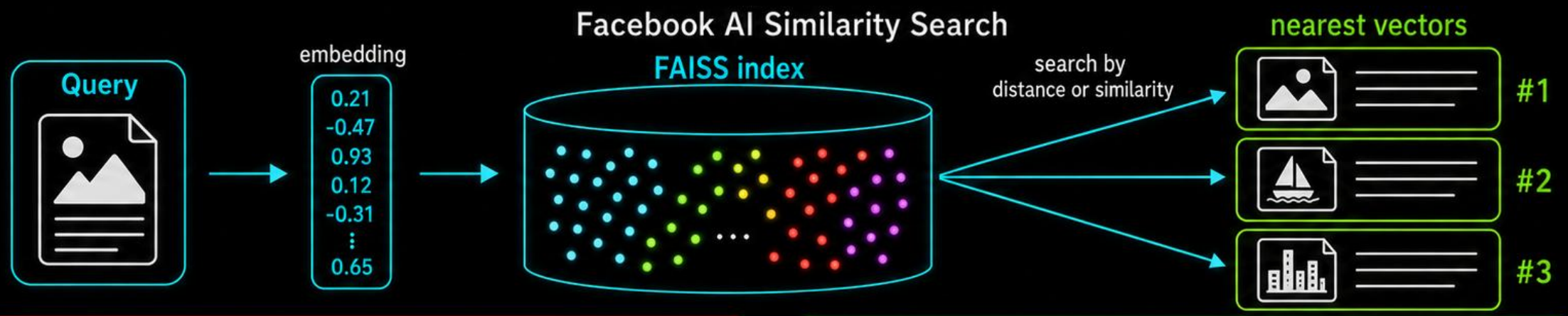


Embedding models learn what 'similar' means; nearest-neighbor search retrieves what is close.

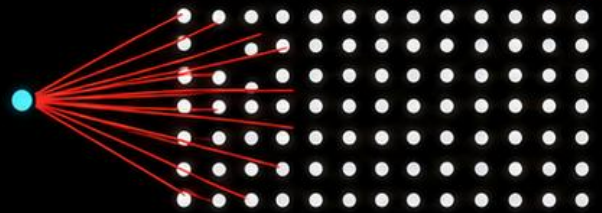
Two Modern Large-Scale Implementations of Nearest Neighbors: FAISS and ScaNN



FAISS



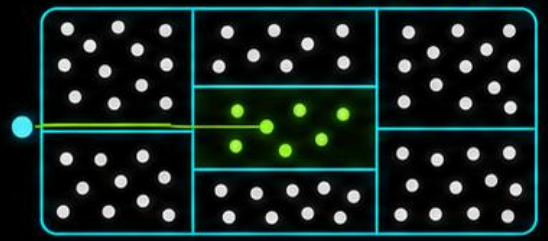
Naive search



Compare with every stored vector

Too slow at large scale

FAISS search



Organize vectors for efficient search

Fast—often approximate

Key trade-off: a tiny loss in exactness can buy a huge gain in speed.

RAG: retrieve relevant passages

Image search: find visual matches

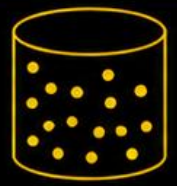
Recommendations: find similar items

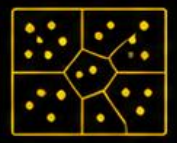
Same nearest-neighbor idea—now operating over millions or billions of embeddings.

ScaNN

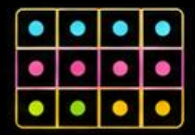
Approximate vector search = prune candidates → score cheaply → re-score accurately

Before queries: build the index





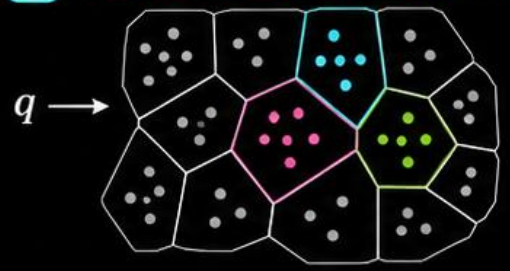
Partition database vectors into regions



Store compressed/quantized representations $\tilde{x}_i$ alongside original vectors x_i

ScaNN's anisotropic quantization penalizes score-changing error more strongly than less consequntial reconstruction error.

1 PRUNE THE SEARCH SPACE



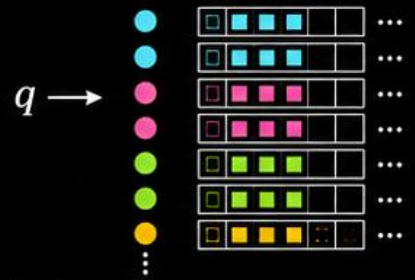
Search only selected partitions

$$C(q) \subset \{1, \ldots, N\}$$

$C(q)$: candidate database indices

far fewer than N vectors

2 APPROXIMATE SCORING



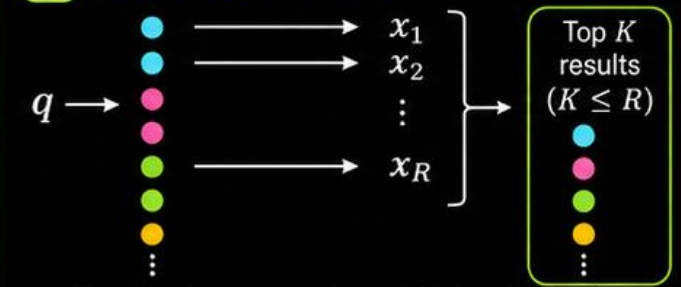
Rank candidates using quantized vectors

$$\tilde{s}_i = q^\top \tilde{x}_i, \quad i \in C(q)$$

Keep the best R candidates

shortlist of size R

3 EXACT RE-SCORING



Recompute scores using original vectors

$$s_i = q^\top x_i$$

Return the top K, where $K \leq R$

Re-scoring corrects some quantization errors

What is approximate—and what is exact?

Approximate: partition selection and compressed-vector scores

Exact within shortlist: final scores $q^\top x_i$

Not guaranteed: the true global top K may have been pruned earlier

Main knobs: partitions searched | candidates scored | shortlist size R

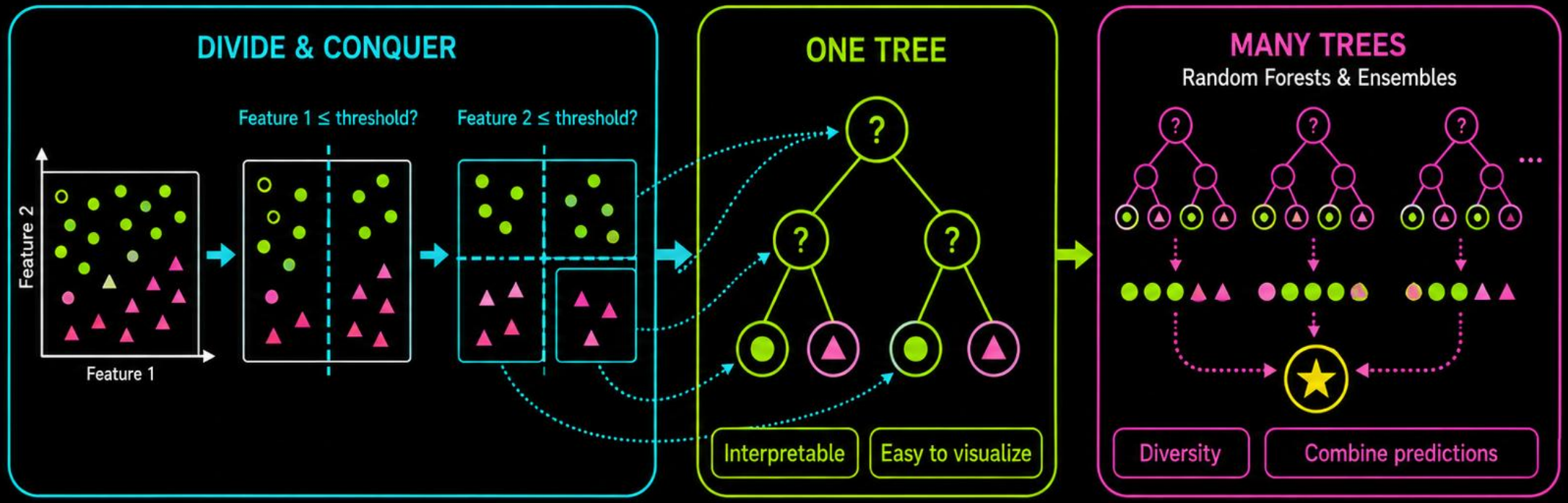
More work → usually higher recall, but greater latency

ScaNN accelerates inner-product search by spending accurate computation only on promising candidates.

Next Class

Computationally Efficient and Powerful Models by Divide & Conquer + Ensembling

Decision Trees • Random Forests • Tree Ensembles



Simple rules + diverse trees → powerful predictions

How can many imperfect trees become a strong learner?